

School/College Tel. No. \_\_\_\_\_ Parents Tel. No. \_\_\_\_\_

[illegible]

## 1] Second smallest Element in an Array

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
int main()
```

```
{
```

```
    int n;
```

```
    printf("Enter no. of elements");
```

```
    scanf("%d", &n);
```

```
    if (n < 2)
```

```
    {
```

```
        printf("Need at least 2 elements");
```

```
        for(int i=0; i<n; i++) return 0;
```

```
    }
```

```
    int a[n];
```

```
    printf("Enter the elements");
```

```
    for (int i=0; i<n; i++)
```

```
        scanf("%d", &a[i]);
```

```
    int smallest = INT_MAX, second = INT_MAX;
```

```
    for (int i=0; i<n; i++)
```

```
    {
```

```
        if (a[i] < smallest)
```

```
        {
```

```
            second = smallest;
```

```
            smallest = a[i];
```

```
        }
```

```
    } else if (a[i] < second && a[i] != smallest)
```

```
    {
```

```
        second = a[i];
```

```
    }
```

```
}
```

```
printf("Second smallest element %d",  
second);
```

```
}
```

2] Sum of left diagonals of matrix

```
#include <stdio.h>
int main()
{
    int n, i, j;
    int matrix[50][50];
    int sum = 0;
    printf("Enter the order of square matrix\n");
    scanf("%d", &n);
    printf("Enter the elements of matrix\n");
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < n; j++)
        {
            scanf("%d", &matrix[i][j]);
        }
    }
    for(i = 0; i < n; i++)
    {
        sum += matrix[i][i];
    }
    printf("Sum of left diagonal elements\n = %d", sum);
}
```

3] Sum of rows and columns of matrix

```
#include <stdio.h>
```

```
int a[50][50], row, col, i, j;
```

```
int sum;
```

```
printf("Enter rows");
```

```
scanf("%d", &row);
```

```
printf("Enter columns");
```

```
scanf("%d", &col);
```

```
printf("Enter elements");
```

```
for(i=0; i<row; i++) {
```

```
for(j=0; j<col; j++)
```

```
{
```

```
scanf("%d", &a[i][j]);
```

```
}
```

```
}
```

```
printf("Sum of rows");
```

```
for(i=0; i<row; i++)
```

```
{
```

```
sum = 0;
```

```
for(j=0; j<col; j++)
```

```
{
```

```
sum += a[i][j];
```

```
}
```

```
printf("Sum of row %d = %d", i+1, sum);
```

```
}
```

```
printf("Sum of columns");
```

```
for(j=0; j<col; j++)
```

```
{
```

```
sum = 0;
```

```
for(i=0; i<row; i++)
```

```
{
```

```
sum += a[i][j];
```

```
}
```

```

printf("Sum of column %d = %d",
j+1, sum);
}
}

```

4] Count total no. of duplicate elements in array

```

#include <stdio.h>
int main()
{
    int a[100], n, i, j, count = 0;
    printf("Enter the no. of elements");
    scanf("%d", &n);
    printf("Enter elements");
    for (i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }
    for (i = 0; i < n; i++)
    {
        for (j = i + 1; j < n; j++)
        {
            if (a[i] == a[j])
            {
                count++;
                break;
            }
        }
    }
    printf("Total number of duplicate elements = %d", count);
}

```

5] Second largest element in an array

```
#include <stdio.h>
int main()
{
    int a[100], n, i;
    int largest, second;
    printf("Enter number of elements");
    scanf("%d", &n);
    printf("Enter the elements");
    for(i=0; i<n; i++)
    {
        scanf("%d", &a[i]);
    }
    largest = second = a[0];
    for(i=1; i<n; i++)
    {
        if(a[i] > largest)
        {
            second = largest;
            largest = a[i];
        }
        else if(a[i] > second && a[i] != largest)
        {
            second = a[i];
        }
    }
    printf("Second largest element = %d", second);
}
```

c) Delete an element at decided position

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a[100], n, pos, i;
```

```
    printf("Enter number of elements");
```

```
    scanf("%d", &n);
```

```
    printf("Enter elements");
```

```
    for(i=0; i<n; i++)
```

```
{
```

```
        scanf("%d", &a[i]);
```

```
}
```

```
    printf("Enter the position to delete");
```

```
    scanf("%d", &pos);
```

```
    if(pos < 1 || pos > n)
```

```
{
```

```
        printf("Invalid position");
```

```
} else {
```

```
    for(i=pos-1; i<n-1; i++)
```

```
{
```

```
        a[i] = a[i+1];
```

```
}
```

```
    n--;
```

```
    printf("Array after deletion");
```

```
    for(i=0; i<n; i++)
```

```
        printf("%d", a[i]);
```

```
}
```

```
}
```

```
}
```

TAT = CT - AT  
WT = TAT - WT

Q C program for FCFS :-

```
#include <stdio.h>
```

```
void swap(int *a, int *b)
```

```
{
```

```
    int temp;
```

```
    temp = *a;
```

```
    *a = *b;
```

```
    *b = temp;
```

```
}
```

```
int main()
```

```
{
```

```
    int i, j, n;
```

```
    int p[20], at[20], bt[20], ct[20],  
        tat[20], wt[20];
```

```
    printf("Enter no. of processes");
```

```
    scanf("%d", &n);
```

```
    for(i=0; i<n; i++)
```

```
{
```

```
        p[i] = i+1;
```

```
        printf("Process %d Arrival time", p[i]);
```

```
        scanf("%d", &at[i]);
```

```
        printf("Process %d Burst time", p[i]);
```

```
        scanf("%d", &bt[i]);
```

```
}
```

```
    for(i=0; i<n-1; i++)
```

```
{
```

```
        for(j=i+1; j<n; j++)
```

```
{
```

```
            if(at[i] > at[j])
```

```
{
```

```
                swap(&at[i], &at[j]);
```

```
                swap(&bt[i], &bt[j]);
```

```

swap(p[i], p[j]);
}
}
ct[0] = at[0] + bt[0]
for(i=1; i<n; i++)
{
    if(ct[i-1] < at[i])
        ct[i] = at[i] + bt[i];
    else
        ct[i] = ct[i-1] + bt[i];
}
for(i=0; i<n; i++)
{
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
}
printf(" P | AT | BT | CT | TAT | WT\n");
for(i=0; i<n; i++)
{
    printf("%d | %d | %d | %d | %d | %d\n",
           p[i], at[i], bt[i],
           ct[i], tat[i], wt[i]);
}
}

```

Output:

Enter number of processes : 3

Process 1 Arrival Time : 0

Process 1 Burst Time : 5

Process 2 Arrival Time : 1

Process 2 Burst Time : 3

Process 3 Arrival Time : 2

Process 3 Burst Time : 4.

| P  | AT | BT | CT | TAT | WT |
|----|----|----|----|-----|----|
| P1 | 0  | 5  | 5  | 5   | 0  |
| P2 | 1  | 3  | 8  | 7   | 4  |
| P3 | 2  | 4  | 12 | 10  | 6  |

Date \_\_\_\_\_  
Page \_\_\_\_\_

1] b] C program for SFS (Non-preemptive)

```

#include <stdio.h>
int main()
{
    int n, i, j;
    int bt[20], wt[20], tat[20], p[20];
    temp;
    printf("Enter no. of processes");
    scanf("%d", &n);
    for (i = 0; i < n; i++)
    {
        p[i] = i + 1;
        printf("Enter Burst Time for P%d", i + 1);
        scanf("%d", &bt[i]);
    }
    for (i = 0; i < n; i++)
    {
        for (j = i + 1; j < n; j++)
        {
            if (bt[i] > bt[j])
            {
                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;
                temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }
    wt[0] = 0,

```

→ Arrived time

```

for(i=1; i<n; i++)
{
    wt[i] = wt[i-1] + bt[i-1];
}
printf("Process | t BT | wt | TAT");
for(i=0; i<n; i++)
{
    tat[i] = wt[i] + bt[i]
    printf("P %d | t %d | t %d | t %d",
           p[i], bt[i], wt[i], tat[i]);
}
}

```

Output:

Enter the number of processes : 4  
 Enter Arrival time for P1 : 0  
 Enter Burst time for P1 : 6  
 Enter Arrival time for P2 : 0  
 Enter Burst time for P2 : 8  
 Enter Arrival time for P3 : 0  
 Enter Burst time for P3 : 7  
 Enter Arrival time for P4 : 0  
 Enter Burst time for P4 : 3

| Process | AT | BT | WT | TAT |
|---------|----|----|----|-----|
| P4      | 0  | 3  | 0  | 3   |
| P1      | 0  | 6  | 3  | 9   |
| P3      | 0  | 7  | 9  | 16  |
| P2      | 0  | 8  | 16 | 24  |

14/3/20

Date \_\_\_\_\_  
Page \_\_\_\_\_

1] C program to simulate the following scheduling algorithm to find turnaround and waiting time.  
Priority (Non-preemptive)

```
#include <stdio.h>
int main()
{
    int n, i, j, pos;
    printf("Enter number of processes");
    scanf("%d", &n);
    int pid[n], at[n], bt[n], pr[n], wt[n],
        tat[n], ct[n];
    int completed[n];
    for(i=0; i<n; i++)
        completed[i] = 0;
    for(i=0; i<n; i++)
    {
        pid[i] = i+1;
        printf("Process %d", i+1);
        printf("Arrival time");
        scanf("%d", &at[i]);
        printf("Burst time");
        scanf("%d", &bt[i]);
        printf("Priority");
        scanf("%d", &pr[i]);
    }
    int time = 0, count = 0;
    while(count < n)
    {
        int highest = 999;
        pos = -1;
```

```

for (i=0; i<n; i++)
{
    if (at[i] <= time && completed[i] == 0)
    {
        if (pr[i] < highest)
        {
            highest = pr[i];
            pos = i;
        }
    }
}

if (pos != -1)
{
    time += bt[pos];
    ct[pos] = time;
    tat[pos] = ct[pos] - at[pos];
    wt[pos] = tat[pos] - bt[pos];
    completed[pos] = 1;
    count++;
}
else
{
    time++;
}
i++;
}

printf("PID\tAT\tBT\tPR\tWT\tTAT");
for (i=0; i<n; i++)
{
    printf("%d\t%d\t%d\t%d\t%d\t%d\n",
           pid[i], at[i], bt[i], pr[i], wt[i],
           tat[i]);
}
}

```

Date \_\_\_\_\_  
Page \_\_\_\_\_

1] c) C program to simulate the following CPU scheduling algorithm to find turnaround and waiting time  
Priority (Preemptive)

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int n, i, time = 0, count = 0, highest;  
printf("Enter number of processes");  
scanf("%d", &n);
```

```
int pid[n], at[n], bt[n], pr[n], rt[n],  
ct[n], wt[n], fat[n];
```

```
for(i=0; i<n; i++)
```

```
{
```

```
pid[i] = i+1;
```

```
printf("Process ", i+1);
```

```
printf("Arrival time");
```

```
scanf("%d", &at[i]);
```

```
printf("Burst time");
```

```
scanf("%d", &bt[i]);
```

```
printf("Priority");
```

```
scanf("%d", &pr[i]);
```

```
rt[i] = bt[i];
```

```
}
```

```
while(count < n)
```

```
{
```

```
highest = -1;
```

```
int best = 999;
```

```
for(i=0; i<n; i++)
```

```
{
```

```
if(at[i] <= time && rt[i] > 0)
```

```
{
```

```

if (pr[i] < best)
{
    best = pr[i];
    highest = 1;
}
}
}
if (highest != -1)
{
    rt[highest]--;
    time++;
    if (rt[highest] == 0)
    {
        count++;
        ct[highest] = time;
        tat[highest] = ct[highest] - at[highest];
        wt[highest] = tat[highest] - bt[highest];
    }
}
else
{
    time++;
}
}
printf("PID\tAT\tBT\tPR\tWT\tTAT");
for (i = 0; i < n; i++)
{
    printf("%d\t%d\t%d\t%d\t%d\t%d",
           pid[i], at[i], bt[i],
           pr[i], wt[i], tat[i]);
}
}

```

# Output for Priority (Non-preemptive)

Enter number of processes : 4

Process 1

Arrival Time : 0

Burst time : 5

Priority : 2

~~PR~~

Process 2

Arrival Time : 1

Burst time : 3

Priority : 1

Process 3

Arrival Time : 2

Burst Time : 8

Priority : 4

Process 4

Arrival Time : 3

Burst time : 6

Priority : 3

| PID | AT | BT | PR | WT | TAI |
|-----|----|----|----|----|-----|
| 1   | 0  | 5  | 2  | 0  | 5   |
| 2   | 1  | 3  | 1  | 4  | 7   |
| 3   | 2  | 8  | 4  | 12 | 20  |
| 4   | 3  | 6  | 3  | 5  | 11  |

## Output for Priority (Preemptive)

Enter number of processes : 4

Process 1

Arrival Time : 0

Burst time : 5

Priority : 2

Process 2

Arrival Time : 1

Burst time : 3

Priority : 8

Process 3

Arrival time : 2

Burst time : 4

Priority : 3

Process 4

Arrival time : 3

Burst time : 2

Priority : 2

| PID | AT | BT | PR | WT | TAT |
|-----|----|----|----|----|-----|
| 1   | 0  | 5  | 2  | 3  | 8   |
| 2   | 1  | 3  | 1  | 0  | 3   |
| 3   | 2  | 4  | 3  | 8  | 12  |
| 4   | 3  | 2  | 2  | 5  | 7   |

13/3/26

# C program for Round Robin

```
#include <stdio.h>
struct Process {
    int id, arrival_time, burst_time, remaining_time,
        completion_time, waiting_time, turnaround_time;
};

int main()
{
    int n, quantum;
    printf("Enter number of processes");
    scanf("%d", &n);
    struct Process p[n];
    for(int i=0; i<n; i++)
    {
        p[i].id = i+1;
        printf("Enter arrival time for process\n %.d", p[i].id);
        scanf("%d", &p[i].arrival_time);
        printf("Enter burst time for process\n %.d", p[i].id);
        scanf("%d", &p[i].burst_time);
        p[i].remaining_time = p[i].burst_time;
        p[i].completion_time = 0;
        p[i].waiting_time = 0;
        p[i].turnaround_time = 0;
    }
    printf("Enter time quantum");
    scanf("%d", &quantum);
    int time = 0, done;
    do {
        done = 1;
        for(int i=0; i<n; i++)
```

```
{
    if (p[i].remaining-time > 0 && p[i].arrival-
        time <= time)
    {
        done = 0;
        if (p[i].remaining-time > quantum)
        {
            time += quantum;
            p[i].remaining-time -= quantum;
        }
        else
        {
            time += p[i].remaining-time;
            p[i].completion-time = time;
            p[i].turnaround-time = p[i].completion-
                time - p[i].arrival-time;
            p[i].waiting-time = p[i].turnaround-time
                - p[i].burst-time;
            p[i].remaining-time = 0;
        }
    }
}

int all-waiting = 1;
for (int i = 0; i < n; i++)
{
    if (p[i].remaining-time > 0 && p[i].arrival-time
        <= time)
    {
        all-waiting = 0;
        break;
    }
}

if (all-waiting) time++;
```

```

} while (!done);
printf("Process AT BT CT TAT WT\n");
int total_wait = 0, total_turn = 0;
for (int i = 0; i < n; i++)
{
    printf("%d %d %d %d %d",
        p[i].id, p[i].arrival-time, p[i].burst-time,
        p[i].arrival-time, p[i].turnaround-time,
        p[i].waiting-time);
    total_wait += p[i].waiting-time;
    total_turn += p[i].turnaround-time;
}

printf("Average waiting time %.2f",
    (float) total_wait / n);
printf("Average Turnaround Time %.2f",
    (float) total_turn / n);

return 0;
}

```

Output:

Enter number of process 3

Enter arrival time for process 1: 0

Enter burst time for process 1: 5

Enter arrival time for process 2: 1

Enter burst time for process 2: 3

Enter arrival time for process 3: 2

Enter burst time for process 3: 4

Enter time quantum: 2

| Process | AT | BT | CT | TAT | WT |
|---------|----|----|----|-----|----|
| P1      | 0  | 5  | 12 | 12  | 7  |
| P2      | 1  | 3  | 9  | 8   | 5  |
| P3      | 2  | 4  | 11 | 9   | 5  |

Average Waiting time: 5.67

Average Turnaround time: 9.67

→ On comparing with diff. quantum times lesser the quantum size makes the system/scheduling algorithm more efficient.

For time quantum = 3

| Process | AT | BT | CT | TAT | WT |
|---------|----|----|----|-----|----|
| P1      | 0  | 5  | 11 | 11  | 6  |
| P2      | 1  | 3  | 6  | 5   | 2  |
| P3      | 2  | 4  | 12 | 10  | 6  |

Avg WT: 4.67

Avg TAT: 8.67

# C program for multilevel with preemption

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
struct Process
{
    int id, arrival-time, burst-time, remaining-time, completion-time;
    char type[10];
    struct Queue {
        int front, rear, size;
        struct Process *arr[100];
    };
    void enqueue(struct Queue *q, struct Process *p)
    {
        q->arr[++q->rear] = p;
        q->size++;
    }
    struct Process* dequeue(struct Queue *q)
    {
        if (q->size == 0) return NULL;
        struct Process* p = q->arr[q->front++];
        q->size--;
        return p;
    }
}

int main()
{
    int n;
    printf("Enter no. of processes");
    scanf("%d", &n);
```

```

struct Process plist[n];
for(int i=0; i<n; i++)
{
    plist[i].id = i+1;
    printf("Enter arrival time for %d",
           plist[i].id);
    scanf("%d", &plist[i].arrival-time);
    printf("Enter burst time for %d",
           plist[i].id);
    scanf("%d", &plist[i].burst-time);
    printf("Enter type (system/user) for
           %d", plist[i].id);
    scanf("%s", plist[i].type);
    plist[i].remaining-time = plist[i].burst-time;
    plist[i].completion-time = 0;
}

for (int i=0; i<n-1; i++)
{
    for(int j=i+1; j<n; j++)
    {
        if(plist[i].arrival-time > plist[j].arrival-time)
        {
            struct Process temp = plist[i];
            plist[i] = plist[j];
            plist[j] = temp;
        }
    }
}

struct Queue systemQ = {0, -1, 0};
struct Queue userQ = {0, -1, 0};
int current-time = 0; completed = 0; i = 0;
while struct Process *current-process =
    NULL;

```

```

while (completed < n)
{
    while (i < n && plist[i].arrival_time <=
           current_time)
    {
        if (strcmp(plist[i].type, "system") == 0)
            enqueue(&systemQ, &plist[i]);
        else
            enqueue(&userQ, &plist[i]);
        i++;
    }

    if (current_process != NULL)
    {
        if (strcmp(current_process->type, "user")
            == 0 && systemQ.size > 0)
        {
            enqueue(&userQ, current_process);
            current_process = NULL;
        }
    }

    if (current_process == NULL)
    {
        if (systemQ.size > 0)
            current_process = dequeue(&systemQ);
        else if (userQ.size > 0)
            current_process = dequeue(&userQ);
        else {
            current_time++;
            continue;
        }
    }

    current_process->remaining_time--;
    current_time++;
}

```

```
if (current → process → remaining_time == 0)
{
    current_process → completion_time =
        current_time;
    printf ("Process p:%d (%.s) completed
            at time %d ",
            current_process → id, current_process → type,
            current_time);
    completed++;
    current_process = NULL;
}
}

printf ("Process  AT  BT  CT  TAT  WT
              Type");
for (int k = 0; k < n; k++)
{
    int tat = plist[k].completion_time -
        plist[k].arrival_time;
    int wt = tat - plist[k].burst_time;
    printf ("%d %d %d %d %d %.s",
        plist[k].id, plist[k].arrival_time, plist[k].burst_time,
        plist[k].completion_time, tat,
        wt, plist[k].type);
}
return 0;
}
```

Output:

Enter number of processes : 4  
 Enter arrival time for P1: 0  
 Enter burst time for P1: 5  
 Enter type (system/user) : system  
 Enter arrival time for P2 : 2  
 Enter burst time for P2 : 3  
 Enter type (system/user) : user  
 Enter arrival time for P3: 1  
 Enter burst time for P3: 4  
 Enter type (system/user) : system  
 Enter arrival time for P4: 3  
 Enter burst time for P4: 2  
 Enter type (system/user) : user.

| Process | type   | AT | BT | CT | WT | TAT |
|---------|--------|----|----|----|----|-----|
| P1      | System | 0  | 5  | 5  | 0  | 5   |
| P2      | User   | 2  | 3  | 12 | 7  | 10  |
| P3      | System | 1  | 4  | 9  | 4  | 8   |
| P4      | User   | 3  | 2  | 2  | 9  | 11  |

Avg WT : 5.00

Avg TAT : 8.5

Chy 24/3/20

- Rate Monotonic
- Earliest-deadline First
- Proportional, scheduling

```
#include <stdio.h>
#include <math.h>
#define MAX 20
```

```
struct Task {
    int id, bt, deadline, period, ct,
        wt, tat;
```

```
};
```

```
double utilization EDF (struct Task
    tasks[], int n)
```

```
{
```

```
    double u = 0;
```

```
    for (int i = 0; i < n; i++)
```

```
        ut = (double) tasks[i].bt / tasks[i].
```

```
        return u;
```

```
}
```

```
double utilization_rms (struct Task tasks[], int n)
```

```
{
```

```
    double u = 0;
```

```
    for (int i = 0; i < n; i++)
```

```
        ut = (double) tasks[i].bt / tasks[i].period;
```

```
    return u;
```

```
}
```

```
double rm_bound (int n)
```

```
{
```

```
    return n * (pow(2.0, 1.0/n) - 1.0)
```

```
}
```

```

void edf(struct Task tasks[], int n)
{
    for(int i = 0; i < n-1; i++)
    {
        for(int j = i+1; j < n; j++)
        {
            if(tasks[i].deadline > tasks[j].deadline)
            {
                struct Task temp = tasks[i];
                tasks[i] = tasks[j];
                tasks[j] = temp;
            }
        }
    }

    printf("\n Rate Monotonic Scheduling");
    printf("\n PID\t BT\t P Period\t CT\t WT\t TAT");

    int time = 0;
    for(int i = 0; i < n; i++)
    {
        time t = tasks[i].bt;
        tasks[i].ct = time;
        tasks[i].tat = tasks[i].ct;
        tasks[i].wt = tasks[i].tat - tasks[i].bt;

        printf("%d %d %d %d %d %d",
            tasks[i].id, tasks[i].bt,
            tasks[i].period, tasks[i].ct,
            tasks[i].wt, tasks[i].tat);
    }

    double u = utilization_rms(tasks, n);

```

```
double bound = rm-bound(n);
printf("Utilization (RMS) = %0.2f", u);
printf("RM Bound = %0.2f", bound);
if (u <= bound)
    printf("Result: Schedulable");
else
    printf("Result: Not Schedulable");
}

int main()
{
    int n;
    printf("Enter no. of processes");
    scanf("%d", &n);
    struct Task tasks[n];
    printf("Enter process details");
    for (int i = 0; i < n; i++)
    {
        tasks[i].id = i + 1;
        printf("Process %d", i + 1);
        printf("Burst time");
        scanf("%d", &tasks[i].bt);
        printf("Deadline");
        scanf("%d", &tasks[i].deadline);
    }
    edf(tasks, n);
    rms(tasks, n);
    return 0;
}
```

## Producer - Consumer Problem -

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define SIZE 5
int buffer[SIZE]
int in = 0, out = 0;
sem_t, empty, full;
pthread_mutex_t mutex;

void * problems produces(void *arg)
{
    int item;
    while(1)
    {
        sleep(2);
        sem_wait(&full);
        pthread_mutex_lock(&mutex);

        buffer[in] = item;
        printf("Produced : %d at %d\n",
               item, in);
        in = (in + 1) % SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
    }
}
```

```
void * consumed(void * arg)
{
    int item;
    while(1)
    {
        sleep(2);
        sem_wait(&full);
        pthread_mutex_lock(&mux);
        item = buffer[out];
        printf("Consumed: %.d from %.d\n",
               item, out);
        out = (out+1) % SIZE;
        pthread_mutex_unlock(&mux);
        sem_post(&empty);
    }
}
```

```
int main()
{
    pthread_t p, c;

    sem_init(&empty, 0, size);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mux, NULL);

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer,
                  NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);
}
```

Output:

Enter no of items to produce: 3

Enter no of item to consume: 2

$p = 0$  at buffer [0].

$p = 1$  at buffer [1]

$p = 2$  at buffer [2]

$c = 0$  from buffer [0]

$c = 1$  from buffer [1]

Tracing:

$in = 0, out = 0, e = 5, full = 0$

Step 1: P 0

$e \rightarrow 5 \rightarrow 4, in = 1, full = 0 \rightarrow 1$

buffer [0] [1] [1] unlock mutex

Step 2: P 1

$e = 3$

stone = 1 at buffer [1]

$in = 2, full = 2$  unlock mutex

buffer [0] [1] [1] [ ]

Step 3:  $p = 2, e = 2$ , shove 2 at buffer [2]

$in = 3, full = 3$

buffer [0] [1] [2] [ ]

Step 4:  $c = 0, full = 2, out = 1, e = 3$

buffer [ ] [1] [2] [ ]

Step 5:  $c = 1, full = 1, out = 2, e = 4$

buffer [ ] [ ] [2] [ ]  
                  ↑  ↑  
                 out in

## Dining Philosopher Problem:-

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#define N 5
sem_t forks[N];

void * philosopher (void *num)
{
    int id = *(int *) num;

    while(1)
    {
        printf("** philosopher %d is thinking", id);
        sleep(1);

        if(id == N-1)
        {
            sem_wait (&forks (id+1) % N);
            sem_wait (&forks[id]);
        } else {
            sem_wait (&forks[id]);
            sem_wait (&forks[(id+1) % N]);
        }

        printf("Philosopher %d is eating", id);
        sleep(2);

        sem_post (&forks[id]);
        sem_post (&forks[(id+1) % N]);
    }
}
```

Date \_\_\_\_\_  
Page \_\_\_\_\_

```
printf("Philosopher %d finished eating",  
}
```

```
int main()  
{
```

```
pthread_t phi[N];  
int ids[N];
```

```
for(int i=0; i<n; i++)
```

```
{
```

```
ids[i] = i;
```

```
pthread = create(sphi[i], NULL,  
philosopher, ids[i]);
```

```
}
```

```
for(int i=0; i<N; i++)
```

```
{
```

```
pthread_join(phi[i], NULL);
```

```
}
```

```
return 0;
```

```
}
```

Output:

Enter no. of philosophers: 3.

ph 0 is thinking

ph 2 is thinking

ph 1 is thinking

ph 0 picked up right fork 1

ph 0 picked up left fork 0

ph 0 is eating

ph 1 picked up right fork 2

ph 0 down forks 0 and 1

## Banker's Algorithm -

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n, m;
```

```
    printf("Enter no. of processes");
```

```
    scanf("%d", &n);
```

```
    printf("Enter no. of resources");
```

```
    scanf("%d", &m);
```

```
    int alloc[n][m], max[n][m], avail[m];
```

```
    printf("Enter allocation matrix");
```

```
    for(int i=0; i<n; i++)
```

```
        for(int j=0; j<m; j++)
```

```
            scanf("%d", &alloc[i][j]);
```

```
    printf("Enter maximum matrix");
```

```
    for(int i=0; i<n; i++)
```

```
        for(int j=0; j<m; j++)
```

```
            scanf("%d", &max[i][j]);
```

```
    printf("Enter available resources");
```

```
    for(int j=0; j<m; j++)
```

```
        scanf("%d", &avail[j]);
```

```
    int need[n][m];
```

```
    for(int i=0; i<n; i++)
```

```
        for(int j=0; j<m; j++)
```

```
            need[i][j] = max[i][j] - alloc[i][j];
```

```
int f[m], ans[m], ind = 0;  
for (int i = 0; i < n; i++) f[i] = 0;
```

```
for (int k = 0; k < n; k++)  
{
```

```
    for (int j = 0; j < m; j++)
```

```
    {
```

```
        if (f[i] == 0)
```

```
        {
```

```
            int flag = 0;
```

```
            for (int j = 0; j < m; j++)
```

```
            {
```

```
                if (need[i][j] & avail[j])
```

```
                {
```

```
                    flag = 1;
```

```
                    break;
```

```
                }
```

```
            }
```

```
            if (flag == 0)
```

```
            {
```

```
                ans[ind++] = i;
```

```
                for (int y = 0; y < m; y++)
```

```
                    avail[y] += alloc[i][y];
```

```
                f[i] = 1;
```

```
            }
```

```
        }
```

```
    }
```

```
    int safe = 1;
```

```
    for (int i = 0; i < n; i++)
```

```
    {
```

```
        if (f[i] == 0)
```

```
        {
```

```
            safe = 0;
```

```
printf("System is not in a safe state");  
break;
```

```
}
```

```
}
```

```
if (safe)
```

```
{
```

```
printf("System is in a safe state");
```

```
for (int i = 0; i < n; i++)
```

```
printf("%d", ans[i]);
```

```
}
```

```
}
```

Output:

Enter no. of processes: 5

Enter no. of resources: 3

Enter Allocation matrix:

0 1 0

2 0 0

3 0 2

2 1 1

0 0 2

Enter maximum matrix

7 5 3

3 2 2

9 0 2

2 2 2

4 3 3

Enter Available resources:

3 3 2

System is in a safe state

Safe sequence: P1 P3 P4 P0 P2

## Deadlock Detection :-

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    printf("Enter no. of processes");
```

```
    scanf("%d", &n);
```

```
    printf("Enter no. of resources");
```

```
    scanf("%d", &m);
```

```
    int alloc[n][m], request[n][m], avail[m];
```

```
    printf("Enter allocation matrix");
```

```
    for(int i=0; i<n; i++)
```

```
        for(int j=0; j<m; j++)
```

```
            scanf("%d", &alloc[i][j]);
```

```
    printf("Enter request matrix");
```

```
    for(int i=0; i<n; i++)
```

```
        for(int j=0; j<m; j++)
```

```
            scanf("%d", &request[i][j]);
```

```
    printf("Enter available resources");
```

```
    for(int j=0; j<m; j++)
```

```
        scanf("%d", &avail[j]);
```

```
    int finish[n];
```

```
    for(int i=0; i<n; i++) finish[i]=0;
```

```
    int work[m];
```

```
    for(int j=0; j<m; j++) work[j]=avail[j];
```

```
    int changed;
```

```
    do {
```

```

changed = 0;
for (int i = 0; i < n; i++)
{
    if (!finish[i])
    {
        int canFinish = 1;
        for (int j = 0; j < m; j++)
        {
            if (request[i][j] > work[j])
            {
                canFinish = 0;
                break;
            }
        }
        if (canFinish)
        {
            for (int j = 0; j < m; j++)
            {
                work[j] += alloc[i][j];
                finish[i] = 1;
                changed = 1;
            }
        }
    }
}
while (changed);
int deadlock = 0;
for (int i = 0; i < n; i++)
{
    if (!finish[i])
    {
        deadlock = 1;
        printf("Process %d is in deadlock", i);
    }
}

```

```

if (!deadlock)
    printf("No deadlock detected");
}

```

Output:

Enter no. of processes: 5

Enter no. of resources: 3

Enter allocation matrix

|   |   |   |
|---|---|---|
| 0 | 1 | 0 |
| 2 | 0 | 0 |
| 3 | 0 | 3 |
| 2 | 1 | 1 |
| 0 | 0 | 2 |

Enter Request Matrix:

|   |   |   |
|---|---|---|
| 0 | 0 | 0 |
| 2 | 0 | 2 |
| 0 | 0 | 0 |
| 1 | 0 | 0 |
| 0 | 0 | 2 |

Enter available resources:

|   |   |   |
|---|---|---|
| 0 | 0 | 0 |
|---|---|---|

No deadlock detected.

## Worst Fit

```
#include <stdio.h>
int main()
{
    int blockSize[20], processSize[20];
    int allocation[20];
    int m, n, i, j;
    printf("Enter no. of memory blocks");
    scanf("%d", &m);
    printf("Enter sizes of memory blocks");
    for(i=0; i<m; i++)
    {
        scanf("%d", &blockSize[i]);
    }
    printf("Enter no. of processes");
    scanf("%d", &n);
    printf("Enter sizes of processes");
    for(i=0; i<n; i++)
    {
        scanf("%d", &processSize[i]);
    }
    for(i=0; i<n; i++)
    {
        allocation[i] = -1;
    }
    for(i=0; i<n; i++)
    {
        int worstIdx = -1;
        for(j=0; j<m; j++)
        {
            if(blockSize[j] >= processSize[i])
            {
                if(worstIdx == -1 || blockSize[j] >
                    blockSize[worstIdx])
                {

```

```

        worstIdx = j;
    }
}
if (worstIdx != -1)
{
    allocation[i] = worstIdx;
    block Size[worstIdx] -= processSize[i];
}
}
printf("Process No, Process Size Block No.");
for (i = 0; i < n; i++)
{
    printf("%d %d", i+1, processSize[i]);
    if (allocation[i] != -1)
        printf("%d", allocation[i]+1);
    else
        printf("Not Allocated");
}
return 0;
}

```

Output:

| Process No. | Process Size | Block No.     |
|-------------|--------------|---------------|
| 1           | 212          | 5             |
| 2           | 417          | 2             |
| 3           | 112          | 5             |
| 4           | 426          | Not allocated |

## Best Fit

```
#include <stdio.h>
int main()
{
    int blockSize[20], processSize[20];
    int allocation[20];
    int m, n, i, j;
    printf("Enter no. of memory blocks");
    scanf("%d", &m);

    printf("Enter sizes of memory blocks");
    for(i=0; i<m; i++)
    {
        scanf("%d", &blockSize[i]);
    }

    printf("Enter no. of process");
    scanf("%d", &n);

    printf("Enter sizes of processes");
    for(i=0; i<n; i++)
    {
        scanf("%d", &processSize[i]);
    }

    for(i=0; i<n; i++)
    {
        allocation[i] = -1;
    }

    for(i=0; i<n; i++)
    {
        int lastIdx = -1;
    }
}
```

```

for (j=0; j<m; j++)
{
    if (blockSize[j] >= processSize[i]);
}
if (leastIdx == -1 || blockSize[j] <
    blockSize[leastIdx])
{
    leastIdx = j;
}
}
}
if (leastIdx != -1)
{
    allocation[i] = leastIdx;
    blockSize[leastIdx] = processSize[i];
}
}
printf("Process No %d Process Size %d\n",
        Block No");
for (i=0; i<n; i++)
{
    printf("%d", allocation[i]+1);
    else
        printf("Not allocated");
}

```

Output:

Enter no. of memory blocks: 5

Enter sizes

100 500 200 800 600

Enter no. of process: 4

Enter sizes of process

212 417 112 426

| Process No | Process Size | Block No |
|------------|--------------|----------|
| 1          | 812          | 4        |
| 2          | 417          | 2        |
| 3          | 112          | 3        |
| 4          | 426          | 5        |

## First fit

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int blockSize[20], processSize[20];
```

```
int allocation[20];
```

```
int m, n, i, j;
```

```
printf("Enter no. of memory blocks");  
scanf("%d", &m);
```

```
printf("Enter sizes of memory block");  
for(i=0; i<m; i++)  
scanf("%d", &blockSize[i]);
```

```
for(i=0; i<n; i++)  
{
```

```
allocation[i] = -1;
```

```
}
```

```
for(i=0; i<n; i++)
```

```
for(j=0; j<m; j++)
```

```
{
```

```
if (blockSize[j] >= processSize[i])
```

```
{
```

```
allocation[i] = j;
```

```
blockSize[j] -= processSize[i];
```

```
break;
```

```
}
```

```
}
```

```

printf("Process No Process Size Block
for(i=0 ; i<n ; i++)
{
    printf("%.d", i+1, processSize[i]);
}
if (allocation[i] != -1)
    printf("%.d", allocation[i]+1);
else
    printf("Not allocated");
}
return 0;
}

```

Output:

Enter no of memory blocks : 5  
 Enter sizes of memory blocks :  
 100 500 200 300 600  
 Enter no of processes : 4  
 Enter sizes of processes  
 212 417 112 426

| Process No. | Process Size | Block No.     |
|-------------|--------------|---------------|
| 1           | 212          | 2             |
| 2           | 417          | 5             |
| 3           | 112          | 2             |
| 4           | 426          | Not allocated |

## First In First out

```
#include <stdio.h>
int main()
{
    int pages[50], frames[10];
    int n, f, i, j, k, pos = 0, faults = 0,
        found;
    printf("Enter number of pages");
    scanf("%d", &n);
    printf("Enter page reference string");
    for(i = 0; i < n; i++)
        scanf("%d", &pages[i]);
    printf("Enter number of frames");
    scanf("%d", &f);
    for(i = 0; i < f; i++)
        frames[i] = -1;
    for(i = 0; i < n; i++)
    {
        found = 0;
        for(j = 0; j < f; j++)
        {
            if(frames[j] == pages[i])
            {
                found = 1;
                break;
            }
        }
        if(found == 0)
        {
            frames[pos] = pages[i];
            pos = (pos + 1) % f;
            faults++;
        }
    }
}
```

```
}  
printf("Page %d →", pages[i]);  
for(k=0; k<f; k++)  
{  
    if(frames[k] != -1)  
        printf("%d", frames[k]);  
    else  
        printf("-");  
}  
}  
printf("\n | Total Page Faults %d\n",  
        faults);  
return 0;  
}
```

Output:

Enter no. of pages: 12  
Enter page reference string:  
1 2 3 4 1 2 5  
1 2 3 4 5  
Enter no. of frames: 3

Page 1 → ~~1~~ + 1 - -  
Page 2 → 1 2 -  
Page 3 → 1 2 3  
Page 4 → 4 2 3  
Page 1 → 4 1 3  
Page 2 → 4 1 2  
Page 5 → 5 1 2  
Page 1 → 5 1 2  
Page 2 → 5 1 2  
Page 3 → 5 3 2  
Page 4 → 5 3 4

Page 5 → 5 3 4

Total page faults = 9.

Least Recently Used Page Replacement (LRU)

```
#include <stdio.h>
```

```
int main() {
```

```
{
```

```
int pages[50], frames[10], time[10];
```

```
int n, f, i, j, k, faults = 0;
```

```
int found, least, pos;
```

```
printf("Enter number of pages");
```

```
scanf("%d", &n);
```

```
printf("Enter page reference string");
```

```
for(i=0; i<n; i++)
```

```
scanf("%d", &pages[i]);
```

```
for(i=0; i<f; i++)
```

```
{
```

```
frames[i] = -1;
```

```
time[i] = 0;
```

```
}
```

```
for(i=0; i<n; i++)
```

```
{
```

```
found = 0;
```

```
for(j=0; j<f; j++)
```

```
{
```

```
if(frames[j] == pages[i])
```

```
{
```

```
found = 1;
```

```

time[j] = i;
break;
}
}
if (found == 0)
{
    least = time[0];
    pos = 0;
    for (j = 1; j < f; j++)
    {
        if (time[j] < least)
        {
            least = time[j];
            pos = j;
        }
    }
    frame[pos] = pages[i];
    time[pos] = i;
    faults++;
}
printf("Page %d →", pages[i]);

for (k = 0; k < f; k++)
{
    if (frames[k] != -1)
        printf("%d", frames[k]);
    else
        printf("-");
}

printf("Total Page Faults = %d", faults);
return 0;
}

```

Output:

Enter number of pages: 12

Enter pages reference string: 1 2 3 4 5

Enter number of frames: 3

Output:

Page 1 → 1 - -

Page 2 → 1 2 -

Page 3 → 1 2 3

Page 4 → 4 2 3

Page 1 → 4 1 3

Page 2 → 4 1 2

Page 5 → 5 1 2

Page 1 → 5 1 2

Page 2 → 5 1 2

Page 3 → 3 1 2

Page 4 → 3 4 2

Page 5 → 3 4 5

Total Page Faults = 10

## Optimal Page Replacement :-

```
#include <stdio.h>
int main()
{
    int pages[50], frames[10];
    int n, f, i, j, k, faults = 0;
    int found, pos, farthest, index;
```

```
    printf("Enter no. of pages");
    scanf("%d", &n);
    printf("Enter page reference string");
    for (i = 0; i < n; i++)
        scanf("%d", &pages[i]);
```

```
    printf("Enter no. of frames");
    scanf("%d", &f);
```

```
    for (i = 0; i < f; i++)
        frames[i] = -1;
```

```
    for (i = 0; i < f; i++)
        frames[i] = 0;
```

```
    for (j = 0; j < f; j++)
```

```
{
```

```
        if (frames[j] == pages[i])
```

```
{
```

```
            found = -1;
```

```
            break;
```

```
}
```

```
}
```

```
if (found == 0)
{
    pos = -1;
    for (j = 0; j < f; j++)
    {
        index = -1;
        for (k = i + 1; k < n; k++)
        {
            if (frames[j] == pages[k])
            {
                index = k;
                break;
            }
        }
        if (index == -1)
        {
            pos = j;
            break;
        }
        if (j == 0 || index > farthest)
        {
            farthest = index;
            pos = j;
        }
    }

    frames[pos] = pages[i];
    faults++;
}

printf ("Page %d -> ", pages[i]);
for (j = 0; j < f; j++)
{
    if (frames[j] != -1)
        printf ("%d ", frames[j]);
    else

```

```

    printf("-");
}
}
printf("Total page faults", faults);
return 0;
}

```

Output:

Enter no. of pages: 12

Enter page reference string:

1 2 3 4 1 2 5 1 2 3 4 5

Enter no. of frames: 3

Page 1 → 1 - -

Page 2 → 1 2 -

Page 3 → 1 2 3

Page 4 → 1 2 4

Page 1 → 1 2 4

Page 2 → 1 2 4

Page 5 → 1 2 5

Page 1 → 1 2 5

Page 2 → 1 2 5

Page 3 → 3 2 5

Page 4 → 4 2 5

Page 5 → 4 2 5

Total Page Faults: 7

22/11/2022